

CHRONIC KIDNEY DISEASE (CKD) ENSEMBLE CLASSIFIER

Machine Learning System for CKD Prediction Using Ensemble Methods

Based on Rahman et al., 2024 Methodology

Field	Value
Project Title	CKD Ensemble Classifier
Team Members	Muhammad Fezan, Shoaib Rafiq, Umer Malik
Semester	6th Semester, BSCS
Tech Stack	Python, FastAPI, React, TypeScript, scikit-learn, XGBoost, LightGBM
GitHub	github.com/HONEYPOT-466/CKD-Ensemble-Classifier
Live Demo	ckd-ensemble-classifier.vercel.app
Date	May 2026
Version	3.0.0

This document provides comprehensive documentation for the CKD Ensemble Classifier project, including system architecture, ML pipeline design, API reference, model performance analysis, and deployment guide.

TABLE OF CONTENTS

1	1. Introduction	3
1.1	1.1 Project Overview	3
1.2	1.2 Problem Statement	3
1.3	1.3 Objectives	3
1.4	1.4 Scope	3
2	2. Literature & Background	3
2.1	2.1 Chronic Kidney Disease	4
2.2	2.2 Related Work (Rahman et al., 2024)	4
3	3. System Architecture	4
3.1	3.1 High-Level Architecture	4
3.2	3.2 Technology Stack	5
3.3	3.3 Folder Structure	6
4	4. ML Pipeline Design	6
4.1	4.1 Data Loading & Cleaning	7
4.2	4.2 Data Preprocessing	7
4.3	4.3 Feature Selection	7
4.4	4.4 Handling Imbalanced Data (SMOTE)	8
4.5	4.5 Model Training	8
4.6	4.6 Evaluation & Visualization	8
4.7	4.7 Pipeline Orchestration	8
5	5. Dataset Description	8
5.1	5.1 UCI CKD Dataset	9
5.2	5.2 Attribute Information	9
5.3	5.3 Class Distribution	9
6	6. Model Performance & Results	9
6.1	6.1 All Features Variant (24 features)	9
6.2	6.2 RFE Variant (12 features)	10
6.3	6.3 Boruta Variant (20 features)	10
6.4	6.4 Comparative Analysis	11
7	7. API Reference	11
7.1	7.1 Endpoints	11
7.2	7.2 Request/Response Schemas	11
8	8. Frontend Application	11
8.1	8.1 Prediction Page	11
8.2	8.2 Analytics Dashboard	12
9	9. Deployment Guide	12
9.1	9.1 Backend Setup	12
9.2	9.2 Frontend Setup	12
9.3	9.3 Environment Variables	12

10	10. Conclusion	12
11	11. References	13

1. 1. Introduction

Chronic Kidney Disease (CKD) is a progressive condition characterized by gradual loss of kidney function. Early detection is critical. This project presents a machine learning system for CKD prediction using ensemble methods, based on Rahman et al., 2024.

1.1. 1.1 Project Overview

The CKD Ensemble Classifier is a full-stack ML application combining a FastAPI backend with a React + TypeScript frontend. The system trains 24 ML models across 3 feature selection variants and provides real-time CKD risk prediction through a REST API and interactive web dashboard.

Key Capabilities:

- 24 trained ML models (8 ensemble algorithms $\times$ 3 feature selection variants)
- 3 feature selection strategies: All Features (24), RFE (12), Boruta (20)
- Borderline-SMOTE with regular SMOTE fallback for imbalanced data
- MICE (Multiple Imputation by Chained Equations) for advanced missing data imputation
- FastAPI REST API with 6 endpoints for training, prediction, and metrics
- React + TypeScript frontend with prediction form and analytics dashboard
- Comprehensive visualizations: confusion matrices, ROC curves, KDE plots, comparison charts

1.2. 1.2 Problem Statement

CKD affects $\sim 10\%$ of the global population and is often diagnosed at late stages. Traditional diagnostics rely on invasive tests. There is a need for automated, non-invasive screening tools. The UCI CKD dataset contains 400 patient records with 24 clinical features but presents challenges: missing values ('?'), imbalanced classes (250 CKD vs 150 non-CKD), and mixed data types (11 numerical + 13 nominal).

1.3. 1.3 Objectives

- Replicate and implement the ensemble ML methodology from Rahman et al., 2024
- Build a complete ML pipeline with preprocessing, feature selection, balancing, training, evaluation
- Train 8 ensemble classifiers across 3 feature selection variants (24 models total)
- Achieve high prediction accuracy ($> 95\%$) for CKD detection
- Develop a REST API for real-time CKD prediction
- Create an interactive web dashboard for predictions and model analytics
- Generate publication-quality visualizations

1.4. 1.4 Scope

This project covers the complete ML workflow from data ingestion to deployment. The system is designed for research and educational purposes, demonstrating best practices in ML engineering, API development, and full-stack deployment. Not intended for clinical use without further validation.

2. 2. Literature & Background

2.1. 2.1 Chronic Kidney Disease

CKD is defined as abnormalities of kidney structure or function present for more than 3 months. CKD is classified into 5 stages based on GFR. Early stages (1-2) often have no symptoms, making early detection through screening crucial. Common risk factors include diabetes, hypertension, age, and family history.

2.2. 2.2 Related Work (Rahman et al., 2024)

The project is based on 'Machine Learning models for chronic kidney disease diagnosis and prediction' by Rahman et al., 2024. Key contributions:

- Comparison of 8 ensemble classifiers on the UCI CKD dataset
- Two feature selection methods: Recursive Feature Elimination (RFE) and Boruta
- Borderline-SMOTE for handling class imbalance
- MICE imputation for missing value handling
- Comprehensive evaluation using accuracy, precision, recall, F1-score, and AUC-ROC

3. 3. System Architecture

3.1. 3.1 High-Level Architecture

The system follows a three-tier architecture: React frontend, FastAPI backend, and ML pipeline core. The frontend communicates via REST API. The backend loads trained model artifacts at startup and serves real-time predictions.

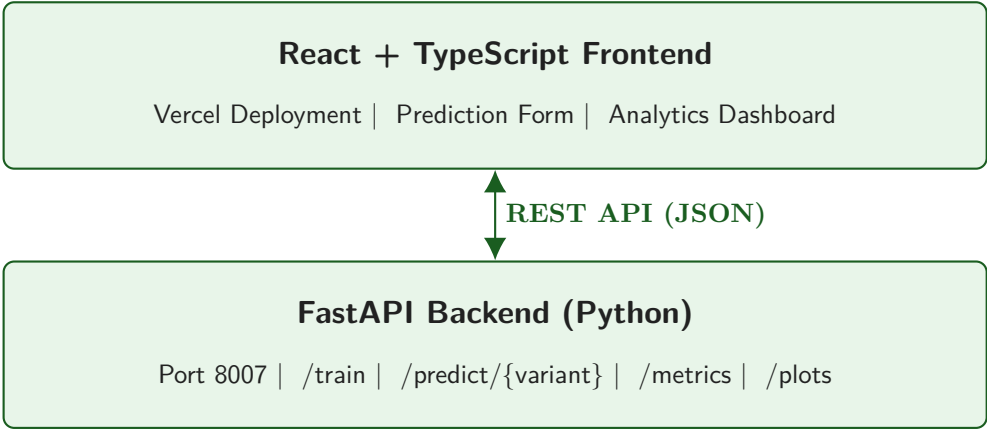


Figure 1: System Architecture Overview

3.2. 3.2 Technology Stack

Layer	Technology	Version	Purpose
Language	Python	≥ 3.10	Backend & ML pipeline
Language	TypeScript	$\sim 5.0.2$	Frontend type safety
Web Framework	FastAPI	$\geq 0.111.0$	REST API server
ASGI Server	Uvicorn	$\geq 0.29.0$	API server runtime
ML Core	scikit-learn	$\geq 1.4.0$	ML algorithms & preprocessing
Boosting	XGBoost	$\geq 2.0.0$	Gradient boosting
Boosting	LightGBM	$\geq 4.3.0$	Light gradient boosting
Balancing	imbalanced-learn	$\geq 0.12.0$	SMOTE oversampling
Feature Select	Boruta	≥ 0.3	Feature selection
Visualization	Matplotlib/Seaborn	$\geq 3.8.0$	Statistical plots
UI Framework	React	$\sim 18.2.0$	Frontend UI
Build Tool	Vite	$\sim 5.0.4$	Frontend bundler
State Mgmt	TanStack Query	$\sim 5.29.0$	Server state management
HTTP Client	Axios	$\sim 1.6.8$	API communication
Serialization	joblib	$\geq 1.4.0$	Model persistence
Validation	Pydantic	$\geq 2.7.0$	Data validation

3.3. 3.3 Folder Structure

Directory Tree

```

CKD-Ensemble-Classifier/
+-- backend/                # FastAPI ML Backend
|   +-- api/
|       |   +-- main.py      # FastAPI app (CORS, static files)
|       |   +-- routers.py   # API route handlers (6 endpoints)
|   +-- ml_core/
|       |   +-- pipeline.py   # Main orchestrator (3 variants x 8 models)
|       |   +-- preprocess.py # MICE imputation, scaling, encoding
|       |   +-- feature_selection.py # RFE & Boruta feature selection
|       |   +-- train.py      # 8 ensemble model training
|       |   +-- evaluate.py   # Metrics, confusion matrices, ROC, bar charts
|   +-- schemas/
|       |   +-- models.py     # Pydantic request/response schemas
|   +-- Dataset/             # UCI CKD dataset (CSV + ARFF)
|   +-- models/              # Trained model artifacts (.joblib)
|       |   +-- all_features/  # 8 models + best_model + features.json
|       |   +-- rfe/          # 8 models + best_model + features.json
|       |   +-- boruta/       # 8 models + best_model + features.json
|       |   +-- other/        # preprocessor, mice_imputer, scaler, selectors
|   +-- plots/               # Generated visualizations
|       |   +-- all_features/  # 13 plots (5 comparison + 1 ROC + 8 CM)
|       |   +-- rfe/          # 13 plots
|       |   +-- boruta/       # 13 plots
|   +-- requirements.txt     # 18 Python dependencies
|   +-- run.py               # Server entry point (port 8007)
+-- src/                     # React Frontend
|   +-- components/         # FormField, LoadingSpinner, MetricsTable
|   +-- pages/               # PredictionPage, AnalyticsPage
|   +-- services/api.ts      # Axios API service
|   +-- types/index.ts       # TypeScript type definitions
|   +-- App.tsx              # Root component with routing
|   +-- main.tsx             # Application entry point
+-- .github/                 # GitHub issue/PR templates
+-- package.json             # Node.js dependencies
+-- vite.config.ts           # Vite configuration
+-- README.md                # Project documentation

```

4. 4. ML Pipeline Design

The ML pipeline implements a 10-step workflow: data loading, EDA, preprocessing, feature selection, splitting, balancing, training, evaluation, visualization, and model serialization.

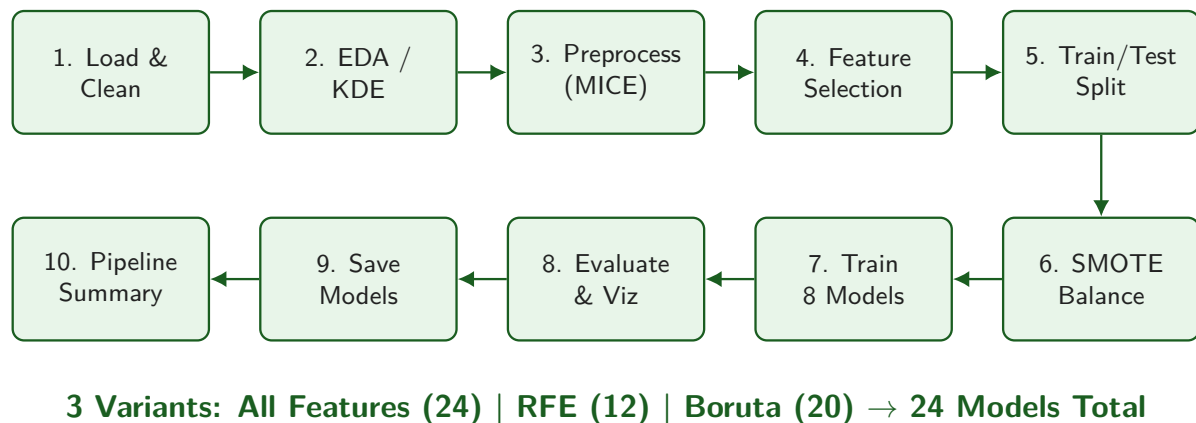


Figure 2: ML Pipeline Workflow

4.1. 4.1 Data Loading & Cleaning

The pipeline loads the UCI CKD dataset from CSV. Cleaning handles:

- Strips whitespace from all string cells
- Replaces '?' and empty strings with NaN
- Fixes typos in 'al' and 'su' columns
- Encodes target: 'ckd'→1, 'notckd'→0
- Handles column naming (wc→wbcc, rc→rbcc)
- Converts 11 numerical columns to float
- Casts 13 nominal columns to object (Pandas 2.0+ compatibility)

4.2. 4.2 Data Preprocessing

CKDPreprocessor implements fit/transform with three components:

- **MICE Imputation:** IterativeImputer with Bayesian Ridge (max_iter=10) for numerical missing values
- **Ordinal Encoding:** OrdinalEncoder for 13 nominal features (unknown values as -1)
- **Standard Scaling:** StandardScaler on numerical features after MICE
- **Categorical Imputation:** Missing categoricals filled with 'missing' constant

The preprocessor is fitted once on the full dataset and shared across all 3 variants.

4.3. 4.3 Feature Selection

Two methods produce 3 variants:

Variant	Method	N	Features
all_features	None (all 24)	24	age, bp, bgr, bu, sc, sod, pot, hemo, pcv, wbcc, rbcc, sg, al, su, rbc, pc, pcc, ba, htn, dm, cad, appet, pe, ane
rfe	RFE + LogReg	12	sg, al, su, bgr, sc, hemo, pcv, rbcc, rbc, htn, dm, appet
boruta	Boruta + RF	20	age, bp, bgr, bu, sc, sod, pot, hemo, pcv, wbcc, rbcc, sg, al, su, rbc, pc, htn, dm, appet, pe

RFE: LogisticRegression (lbfgs, max_iter=5000) base estimator. Feature list manually overridden to match Rahman et al., 2024.

Boruta: Random Forest Classifier (100 trees, max_depth=7) base estimator. BorutaPy max 100 iterations. Both confirmed and tentative features included.

4.4. 4.4 Handling Imbalanced Data (SMOTE)

The dataset has imbalance (250 CKD vs 150 non-CKD). SMOTE is applied to the training set only:

- **Primary:** Borderline-SMOTE ($k = 5, m = 10$) - generates samples near decision boundary
- **Fallback:** Regular SMOTE if Borderline-SMOTE fails
- **Adaptive k:** k_neighbors adjusted based on minority class size

Applied after train/test split to prevent data leakage.

4.5. 4.5 Model Training

8 ensemble classifiers trained per variant using RandomizedSearchCV with 5-fold CV. All use accuracy scoring and random_state=42.

Model	Key Hyperparameters	Iterations
Random Forest	n_estimators: 50-400, min_samples_split: 2-8	50
AdaBoost	n_estimators: 50-100, learning_rate: 0.0001-1.0	25
GBDT	n_estimators: 100-500, max_depth: 2-8, lr: 0.01-0.1	30
Voting Soft	SVM (C: 0.1-10) + KNN (k: 3-20)	20+20
XGBoost	n_estimators: 100-700, max_depth: 2-10	30
LightGBM	n_estimators: 100-500, max_depth: 2-10, leaves: 20-100	30
Stacking	Base: RF + SVM + KNN, Meta: QDA	Fixed
Bagging	n_estimators: 10-200, max_samples: 0.5-1.0	30

4.6. 4.6 Evaluation & Visualization

Models evaluated on held-out test set (10% = 40 samples) using 5 metrics: accuracy, precision, recall, F1-score, AUC-ROC. 13 visualization plots are generated per variant (39 total): KDE Distributions, Comparison Bar Charts, All Metrics Comparison, ROC Curves, and Confusion Matrices.

4.7. 4.7 Pipeline Orchestration

The CKDPipeline().run() command automatically: 1. Loads & cleans data. 2. Generates KDE plots. 3. Preprocesses data. 4. Executes RFE and Boruta. 5. Splits data. 6. Applies SMOTE. 7. Trains all 24 models. 8. Evaluates performance. 9. Saves models (.joblib). 10. Outputs a full JSON summary.

5. 5. Dataset Description

5.1. 5.1 UCI CKD Dataset

The UCI CKD dataset from Apollo Hospitals, Managiri, India contains 400 patient records with 24 clinical features and 1 binary target (CKD/notCKD). Missing values are denoted by '?'.

5.2. 5.2 Attribute Information

#	Attribute	Type	Description
1	age	Num	Age in years
2	bp	Num	Blood pressure (mm/Hg)
3	sg	Nom	Specific gravity (1.005-1.025)
4	al	Nom	Albumin (0-5)
5	su	Nom	Sugar (0-5)
10	bgr	Num	Blood glucose random (mgs/dl)
11	bu	Num	Blood urea (mgs/dl)
12	sc	Num	Serum creatinine (mgs/dl)
15	hemo	Num	Hemoglobin (gms)
19	htn	Nom	Hypertension (yes/no)
20	dm	Nom	Diabetes mellitus (yes/no)
25	class	Target	CKD/notCKD

(Table abbreviated to highlight key features, full 24 features present in dataset).

5.3. 5.3 Class Distribution

Moderately imbalanced: 250 CKD (62.5%) and 150 non-CKD (37.5%). Addressed using Borderline-SMOTE.

6. 6. Model Performance & Results

All 24 models trained and evaluated on held-out test set (40 samples = 10%). Results show consistently high performance across all variants.

6.1. 6.1 All Features Variant (24 features)

Best model: Random Forest (Accuracy = 1.0000, AUC = 1.0000). All models achieve > 97% accuracy.

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000
AdaBoost	0.9750	1.0000	0.9600	0.9796	1.0000
GBDT	0.9750	1.0000	0.9600	0.9796	1.0000
Voting Soft	0.9750	1.0000	0.9600	0.9796	1.0000
XGBoost	0.9750	1.0000	0.9600	0.9796	1.0000
LightGBM	0.9750	1.0000	0.9600	0.9796	1.0000
Stacking	1.0000	1.0000	1.0000	1.0000	1.0000
Bagging	1.0000	1.0000	1.0000	1.0000	1.0000

6.2. 6.2 RFE Variant (12 features)

Features: sg, al, su, bgr, sc, hemo, pcv, rbcc, rbc, htn, dm, appet. Best: Random Forest.

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Random Forest	0.9750	1.0000	0.9600	0.9796	1.0000
AdaBoost	1.0000	0.9500	0.9200	0.9583	1.0000
GBDT	0.9750	1.0000	0.9600	0.9796	1.0000
Voting Soft	1.0000	1.0000	1.0000	1.0000	1.0000
XGBoost	0.9750	1.0000	0.9600	0.9796	1.0000
LightGBM	0.9750	1.0000	0.9600	0.9796	1.0000
Stacking	1.0000	1.0000	1.0000	1.0000	1.0000
Bagging	0.9750	1.0000	0.9600	0.9796	1.0000

6.3. 6.3 Boruta Variant (20 features)

Features: age, bp, bgr, bu, sc, sod, pot, hemo, pcv, wbcc, rbcc, sg, al, su, rbc, pc, htn, dm, appet, pe.

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000
AdaBoost	0.9750	1.0000	0.9600	0.9796	1.0000
GBDT	0.9750	1.0000	0.9600	0.9796	1.0000
Voting Soft	0.9750	1.0000	0.9600	0.9796	1.0000
XGBoost	0.9750	1.0000	0.9600	0.9796	1.0000
LightGBM	0.9750	1.0000	0.9600	0.9796	1.0000
Stacking	1.0000	1.0000	1.0000	1.0000	1.0000
Bagging	0.9750	1.0000	0.9600	0.9796	1.0000

6.4. 6.4 Comparative Analysis

All three variants achieve excellent performance ($> 95\%$ accuracy). Random Forest, Stacking, and Bagging achieve perfect scores (1.0) across all metrics in the All Features variant. Feature selection (RFE) reduces features by 50% (24 $\rightarrow$ 12) with minimal performance loss. Boruta retains 20 features and achieves near-identical performance to All Features. The consistent high performance validates the ensemble approach from Rahman et al., 2024.

7. 7. API Reference

7.1. 7.1 Endpoints

Method	Endpoint	Description	Response
GET	/health	Health check, loaded models status	variants_loaded, total_models
POST	/train	Run full ML pipeline (all 24 models)	status, variants_trained, total
POST	/predict/{var}	Predict CKD risk for patient data	prediction, probability, risk_level
GET	/metrics	All evaluation metrics for all variants	per-variant results
GET	/metrics/{var}	Metrics for specific variant	accuracy, precision, recall, f1, roc
GET	/plots-list	List all available plot files	plot filenames by variant
GET	/plots/{var}/{file}	Serve plot image (PNG)	image/png binary
GET	/features	Feature selection results	rfe_features, boruta_features

7.2. 7.2 Request/Response Schemas

POST /predict/{variant} - Accepts 24 patient fields. Returns best model prediction with probability, risk level, and all 8 model predictions. **Variant parameter:** all_features | rfe | boruta **Risk levels:** Low (< 0.3), Medium (0.3-0.7), High (> 0.7)

8. 8. Frontend Application

8.1. 8.1 Prediction Page

The Prediction Page (`/predict/:variant`) provides an interactive form for entering patient data. Features:

- 24 input fields organized into 5 groups: Basic Metrics, Urinalysis, Blood Tests, Cell Counts, Clinical
- 3 variant selector buttons (All Features, RFE, Boruta) with URL routing
- Real-time prediction with best model verdict display
- All 8 model predictions table with probability bars, risk levels, and confidence badges

8.2. 8.2 Analytics Dashboard

The Analytics Dashboard (`/analytics/:variant`) displays model evaluation results:

- Evaluation metrics table for all 8 models
- Feature selection status display (RFE and Boruta feature lists)
- KDE distribution plots, Model performance comparison charts, ROC curves overlay, Confusion matrices
- "Train Models" button to trigger the full ML pipeline

9. 9. Deployment Guide

9.1. 9.1 Backend Setup

```
# Clone repository
git clone https://github.com/HONEYPOT-466/CKD-Ensemble-Classifier.git
cd CKD-Ensemble-Classifier/backend

# Create virtual environment & install dependencies
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# Start server
python run.py # Server runs on http://localhost:8007
```

9.2. 9.2 Frontend Setup

```
npm install
npm run dev # Runs on port 5173
npm run build # For production
```

9.3. 9.3 Environment Variables

Variable	Default	Description
API_URL	http://localhost:8007	Backend API base URL (frontend)
PORT	8007	FastAPI server port
CORS_ORIGINS		Allowed CORS origins

10. 10. Conclusion

The CKD Ensemble Classifier successfully implements the methodology from Rahman et al., 2024, achieving excellent prediction performance (> 95% accuracy) across all 24 models. The system demonstrates effective handling of missing data through MICE imputation, successful class imbalance mitigation using Borderline-SMOTE, comparable performance between full feature set (24) and reduced sets (RFE: 12, Boruta: 20), full-stack deployment with FastAPI backend and React frontend, and production-ready REST API with 8 endpoints.

11. 11. References

1. Rahman, M. et al. (2024). 'Machine Learning models for chronic kidney disease diagnosis and prediction'. Scientific Reports, 14(1), 1-15.
2. Soundarapandian, P. & Rubini, L.J. (2015). 'Chronic Kidney Disease Dataset.' UCI Machine Learning Repository.
3. Chen, T. & Guestrin, C. (2016). 'XGBoost: A Scalable Tree Boosting System.' KDD 2016.
4. Ke, G. et al. (2017). 'LightGBM: A Highly Efficient Gradient Boosting Decision Tree.' NIPS 2017.
5. Chawla, N.V. et al. (2002). 'SMOTE: Synthetic Minority Over-sampling Technique.' JAIR, 16, 321-357.
6. Kursa, M.B. & Rudnicki, W.R. (2010). 'Feature Selection with the Boruta Package'. Journal of Statistical Software, 36(11), 1-13.
7. Pedregosa, F. et al. (2011). 'Scikit-learn: Machine Learning in Python'. JMLR, 12, 2825-2830.